Programación Dinámica

Mateo Valdivia

Definición

Es una técnica algorítmica que se basa en un estado inicial del problema y una transición recurrente.

Resuelve subproblemas para resolver el problema.

1. Contar algo (numero de formas de hacer algo)
2. Minimizar o maximizar algo
3. YES/NO questions

Fibonacci

$$F_n = F_{n-2} + F_{n-1} \, (para\ n \geq 2)$$

$$F_0 = 0$$

$$F_1 = 1$$

0,1,1,2,3,5,8...

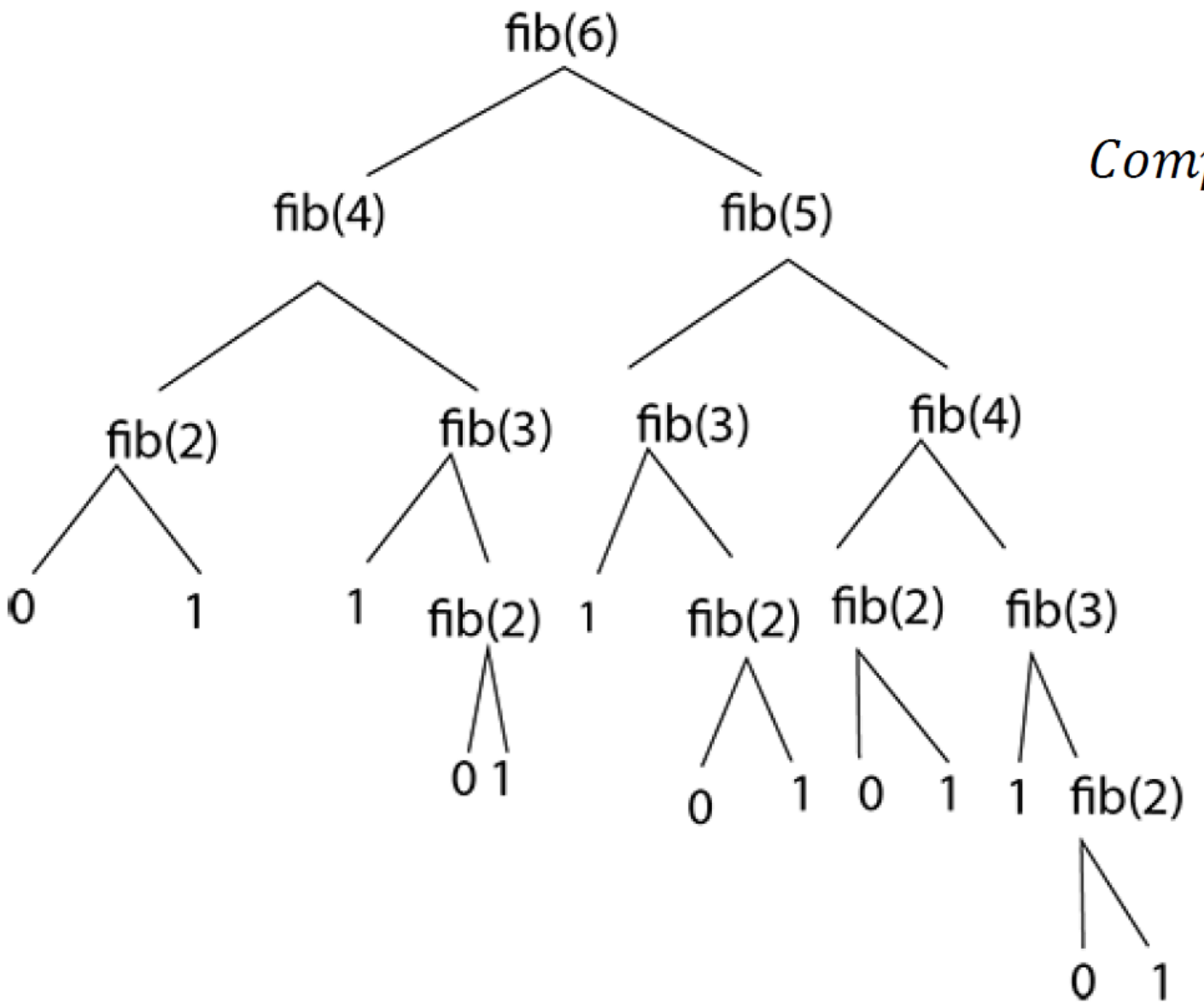
Fibonacci

$$F_n = F_{n-2} + F_{n-1} \quad (para\ n \geq 2)$$
$$F_0 = 0$$
$$F_1 = 1$$

0,1,1,2,3,5,8...

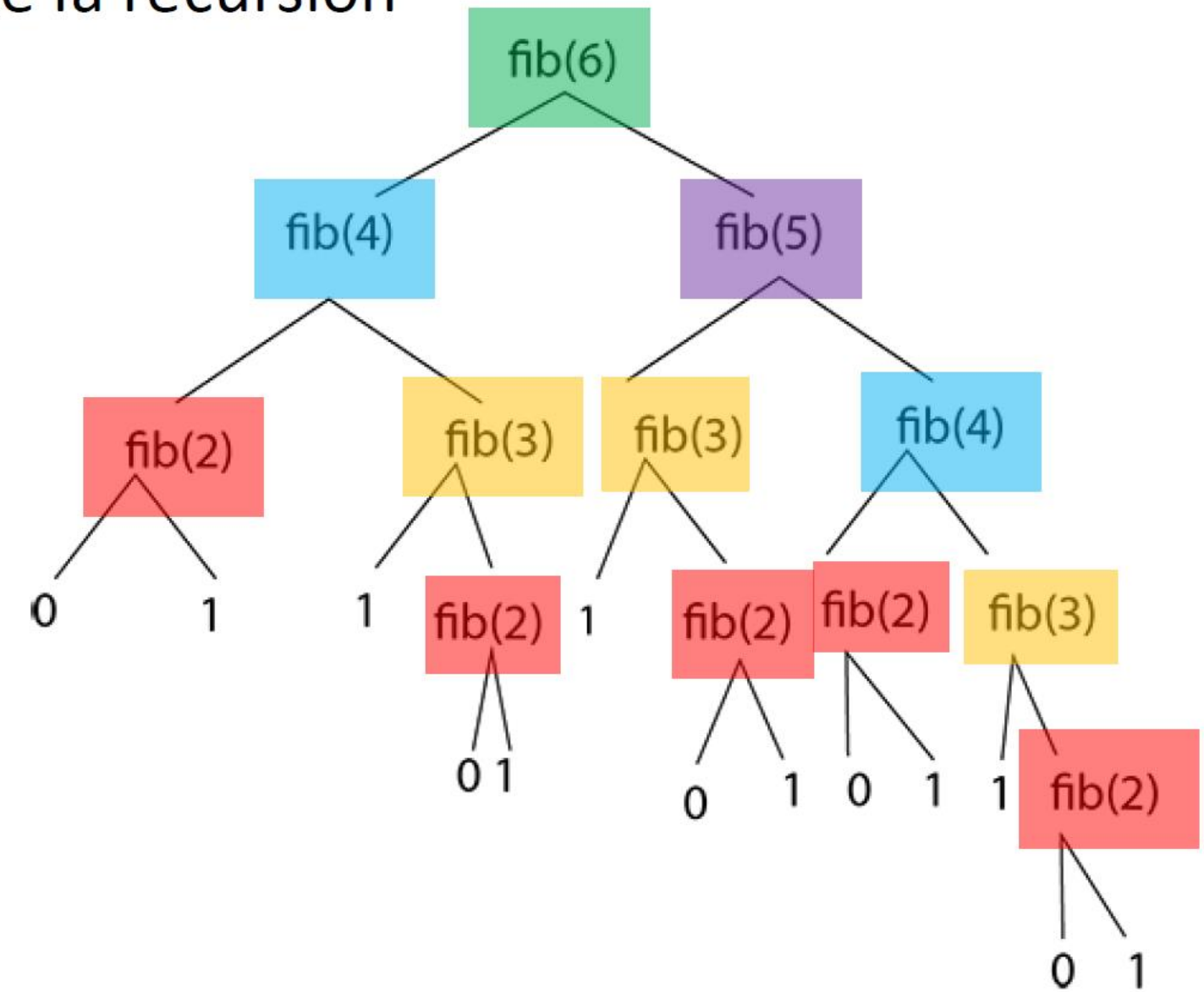
```
int fib(int n){
    if(n==0)return 0;
    if(n==1)return 1;
    return fib(n-1)+fib(n-2);
}
```

Árbol de la recursión

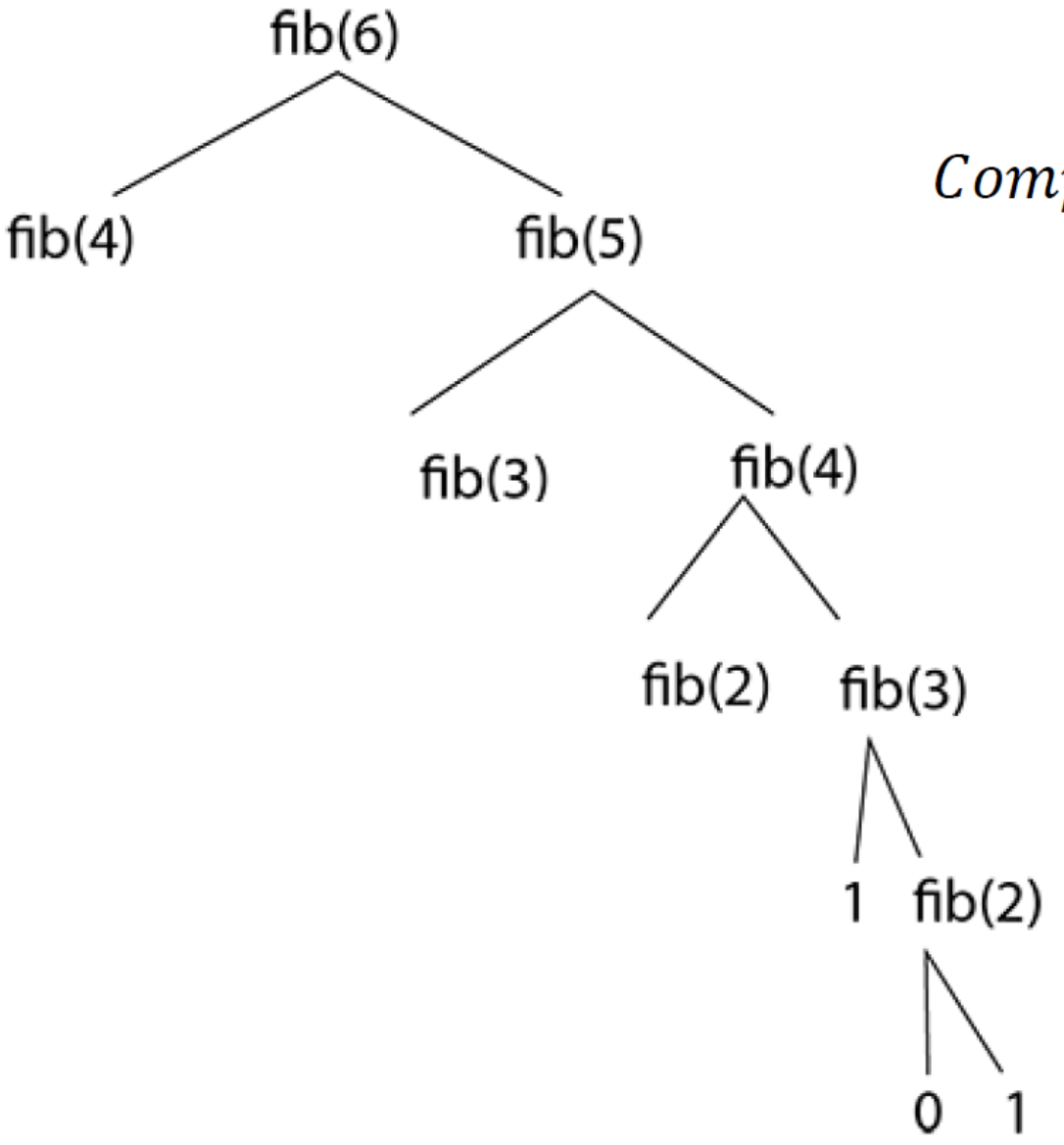


$Complejidad\ O(2^n)$

Árbol de la recursión



Árbol de la recursión



$Complejidad\ O(n)$

Aplico memoización

$O(2^n)$

```
int fib(int n){
    if(n==0)return 0;
    if(n==1)return 1;
    return fib(n-1)+fib(n-2);
}
```

$O(n)$

```
const int tam=505;
int dp[tam];//al inicio lo lleno de menos -1
int fib(int n){
    if(n==0)return 0;
    if(n==1)return 1;
    if(dp[n]!=-1)return dp[n];
    dp[n] = fib(n-1)+fib(n-2);
    return dp[n];
}
```

Iterativo vs Recursivo

$O(n)$

```
int dp[tam];
dp[0]=0;dp[1]=1;
for(int i=2;i<=tam;i++){
    dp[i]=dp[i-1]+dp[i-2];
}
//dp[i]=fibonacci(i)
```

$O(n)$

```
const int tam=505;
int dp[tam];//al inicio lo lleno de menos -1
int fib(int n){
    if(n==0)return 0;
    if(n==1)return 1;
    if(dp[n]!=-1)return dp[n];
    dp[n] = fib(n-1)+fib(n-2);
    return dp[n];
}
```

Problem Statement

There are N stones, numbered $1, 2, \ldots, N$. For each i ($1 \leq i \leq N$), the height of Stone i is h_i.

There is a frog who is initially on Stone 1. He will repeat the following action some number of times to reach Stone N:

- If the frog is currently on Stone i, jump to Stone $i+1$ or Stone $i+2$. Here, a cost of $|h_i - h_j|$ is incurred, where j is the stone to land on.

Find the minimum possible total cost incurred before the frog reaches Stone N.

Constraints

- All values in input are integers.
- $2 \leq N \leq 10^5$
- $1 \leq h_i \leq 10^4$

Sample Input 1 Copy

```
4
10 30 40 20
```

Sample Output 1 Copy

```
30
```

If we follow the path $1 \rightarrow 2 \rightarrow 4$, the total cost incurred would be $|10-30| + |30-20| = 30$.

https://atcoder.jp/contests/dp/tasks/dp_a

Primero resolvemos el problema sin dp (memoización)

```
int n;
vector<int> h;
const int INF=1e9+7;
int f(int pos){
    if(pos==n-1)return 0;//caso base
    int res=abs(h[pos]-h[pos+1])+f(pos+1);//salto a piedra i+1
    if(pos+2<n)res=min(res,abs(h[pos]-h[pos+2])+f(pos+2));//salto a piedra i+2
    return res;
}
int main(){
    cin>>n;
    int x;
    for(int i=0;i<n;i++){
        cin>>x;
        h.push_back(x);
    }
    cout<<f(0)<<endl;
    return 0;
}
```

Ahora aplicamos memoización (dp) que consiste en crear un arreglo con las dimensiones de los parámetros de la función recursiva para no calcular varias veces lo mismo.

```
int n;
vector<int> h;
const int INF=1e9+7;
const int tam=1e5+5;
int dp[tam];
int f(int pos){
    if(pos==n-1)return 0;//caso base
    if(dp[pos]!=-1)return dp[pos];
    dp[pos]=abs(h[pos]-h[pos+1])+f(pos+1);//salto a piedra i+1
    if(pos+2<n)dp[pos]=min(dp[pos],abs(h[pos]-h[pos+2])+f(pos+2));//salto a piedra i+2
    return dp[pos];
}
int main(){
     memset(dp,-1,sizeof(dp));
    cin>>n;
    int x;
    for(int i=0;i<n;i++){
        cin>>x;
        h.push_back(x);
    }
    cout<<f(0)<<endl;
    return 0;
}
```

Problema 2

Your task is to count the number of ways to construct sum n by throwing a dice one or more times. Each throw produces an outcome between 1 and 6.

For example, if $n = 3$, there are 4 ways:

- $1 + 1 + 1$
- $1 + 2$
- $2 + 1$
- 3

Input

The only input line has an integer n.

Output

Print the number of ways modulo $10^9 + 7$.

Constraints

- $1 \leq n \leq 10^6$

Example

Input:
3

Output:
4

https://cses.fi/problemset/task/1633

Solución sin memoización (dp) TLE

```
int n;
const int MOD=1e9+7;
long long f(int suma){
    if(suma==n)return 1;
    if(suma>n)return 0;
    long long res=0;
    for(int i=1;i<=6;i++){
        res+=f(suma+i);
        res%=MOD;
    }
    return res;
}
int main(){
    cin>>n;
    cout<<f(0)<<endl;
    return 0;
}
```

Solución con dp AC

```
int n;
const int MOD=1e9+7;
const int tam=1e6+5;//tamaño maximo de n
long long dp[tam];
long long f(int suma){
    if(suma==n)return 1;
    if(suma>n)return 0;
    if(dp[suma]!=-1)return dp[suma];
    dp[suma]=0;
    for(int i=1;i<=6;i++){
        dp[suma]+=f(suma+i);
        dp[suma]%=MOD;
    }
    return dp[suma];
}
int main(){
    memset(dp,-1,sizeof(dp));
    cin>>n;
    cout<<f(0)<<endl;
    return 0;
}
```

Problem Statement

Taro's summer vacation starts tomorrow, and he has decided to make plans for it now.

The vacation consists of N days. For each i ($1 \leq i \leq N$), Taro will choose one of the following activities and do it on the i-th day:

- A: Swim in the sea. Gain a_i points of happiness.
- B: Catch bugs in the mountains. Gain b_i points of happiness.
- C: Do homework at home. Gain c_i points of happiness.

As Taro gets bored easily, he cannot do the same activities for two or more consecutive days.

Find the maximum possible total points of happiness that Taro gains.

Constraints

- All values in input are integers.
- $1 \leq N \leq 10^5$
- $1 \leq a_i, b_i, c_i \leq 10^4$

Input

Input is given from Standard Input in the following format:

$$
\begin{array}{lll}
N & & \\
a_1 & b_1 & c_1 \\
a_2 & b_2 & c_2 \\
: & & \\
a_N & b_N & c_N
\end{array}
$$

Sample Input 1 Copy

```
3
10 40 70
20 50 80
30 60 90
```

Sample Output 1 Copy

```
210
```

https://atcoder.jp/contests/dp/tasks/dp_c

Problem Statement

Taro's summer vacation starts tomorrow, and he has decided to make plans for it now.

The vacation consists of N days. For each i ($1 \leq i \leq N$), Taro will choose one of the following activities and do it on the i-th day:

- A: Swim in the sea. Gain a_i points of happiness.
- B: Catch bugs in the mountains. Gain b_i points of happiness.
- C: Do homework at home. Gain c_i points of happiness.

As Taro gets bored easily, he cannot do the same activities for two or more consecutive days.

Find the maximum possible total points of happiness that Taro gains.

Constraints

- All values in input are integers.
- $1 \leq N \leq 10^5$
- $1 \leq a_i, b_i, c_i \leq 10^4$

Input

Input is given from Standard Input in the following format:

$$
\begin{array}{l}
N \\
a_1 \quad b_1 \quad c_1 \\
a_2 \quad b_2 \quad c_2 \\
\vdots \\
a_N \quad b_N \quad c_N
\end{array}
$$

Sample Input 1 Copy

```
3
10 40 70
20 50 80
30 60 90
```

Sample Output 1 Copy

```
210
```

https://atcoder.jp/contests/dp/tasks/dp_c

Knapsack Problems

Se refiere a problemas donde se debe rellenar algún contenedor con algún subset de objetos que te dan en la entrada. Digamos una mochila con cierta capacidad y objetos que quieres meter en la mochila sin que se rompa.

- 0/1 Knapsack : Escoger un subset de elementos tal que se el valor sea máximo y se pueda cargar el peso.
- La cantidad de pesos diferentes que se puede lograr sin exceder la capacidad del contenedor.
- Contar la cantidad de secuencias de elementos que llenen el contenedor completamente.

0/1 knapsack

Hay n objetos en un museo cada objeto tiene valor (ai) y peso (pi). Un ladrón tiene una mochila con capacidad w y quiere maximizar el valor de los objetos que robe.

Limites

W<=1e4

N<=1e3

ai<=1e9

pi<=1e5

Input:

```
4 10
4 8 5 3
5 12 8 1
```

Output:

```
13
```

Observaciónes y solución

- No importa en que orden tome los objetos, entonces puedo decidir que objetos robar moviéndome de izquierda a derecha.
- Para cada objeto decido tomarlo o no tomarlo.
- El único limitante para meter un objeto es el peso que carga la mochila actualmente.

Observaciones y solución

```
int dp[105][10005];//dp[pos][capacidad restante] = valor maximo que puedo robar
int n,w;
vi p,a;
int f(int pos, int capacidad){
    if(pos==n)return 0;
    if(dp[pos][capacidad]!=-1)return dp[pos][capacidad];
    dp[pos][capacidad]=f(pos+1,capacidad);
    if(capacidad-p[pos]>=0)dp[pos][capacidad]=max(dp[pos][capacidad],f(pos+1,capacidad-p[pos])+a[pos]);
    return dp[pos][capacidad];
}
int main(){
    memset(dp,-1,sizeof(dp));
    cin>>n>>w;
    int x;
    for(int i=0;i<n;i++){
        cin>>x;p.pb(x);
    }
    for(int i=0;i<n;i++){
        cin>>x;a.pb(x);
    }
    cout<<f(0,w)<<endl;

    return 0;
}
```

Longest Common Subsequence

Given two strings **S** & **T**, find the Longest Common Subsequence.

Let a subsequence of S is the result of deleting some(0 to All) characters in S.

E.g. **S** = "abcde"; "", "abcde", "ae" are the subsequences of **S**, but "aeb" is not.

Remind that **Subsequence != Substring.** (Substring is contiguous)

If S = "abcdef", T = "ebbdaf",

There Longest Common Subsequence = "bdf"

https://leetcode.com/problems/longest-common-subsequence/description/

Longest Common Subsequence

(1-based)

Let **dp[i][j]** = the length of Longest Common Subsequence of $\mathbf{S_1 \ldots S_i}$ and $\mathbf{T_1 \ldots T_j}$.

If **S[i] != T[j]**, we consider **dp[i-1][j]** and **dp[i][j-1]**.

I.e. **dp[i][j]** = max(dp[i-1][j], dp[i][j-1]).

If **S[i] == T[j]**, other then consider **dp[i-1][j]** and **dp[i][j-1]**, we may add this character to the Longest Common Subsequence of $\mathbf{S_1 \ldots S_{i-1}}$ and $\mathbf{T_1 \ldots T_{j-1}}$.

I.e. **dp[i][j]** = max(dp[i-1][j], dp[i][j-1], dp[i-1][j-1] + 1).

dp[0][j] = dp[i][0] = 0 for all i, j because of empty string.

https://leetcode.com/problems/longest-common-subsequence/description/

Range DP

You are given a string consisting of n characters between a and z.

On each turn, you may remove any two adjacent characters that are equal. Your goal is to construct an empty string by removing all the characters.

In how many ways can you do this?

Input

The only input line has a string of length n.

Output

Print one integer: the number of ways modulo $10^9 + 7$.

Constraints

- $1 \leq n \leq 500$

Example

Input:
```
aabccb
```

Output:
```
3
```

https://cses.fi/problemset/task/1080

Range DP

You are given a string consisting of n characters between a and z.

On each turn, you may remove any two adjacent characters that are equal. Your goal is to construct an empty string by removing all the characters.

In how many ways can you do this?

Input

The only input line has a string of length n.

Output

Print one integer: the number of ways modulo $10^9 + 7$.

Constraints

- $1 \leq n \leq 500$

Example

Input:
```
aabccb
```

Output:
```
3
```

https://cses.fi/problemset/task/1080